Search in course

Search

English

B

1. CS 498: Cloud Computing Applications
2. Week 16
3. Machine Problem 13 : AutoML Pipeline

PreviousNext

Practice Programming Assignment: Machine Problem 13 : AutoML Pipeline

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

The MP13 pipeline automates end-to-end fraud-detection model retraining and deployment by orchestrating AWS Step Functions, Lambda, S3, and SageMaker. As soon as new raw data is uploaded into the designated S3 “raw” folder, a trigger Lambda function constructs and gathers all necessary metadata, and starts a Step Functions state machine to coordinate the subsequent steps.

The Step Functions workflow first initiates a SageMaker Processing job to preprocess the incoming data and detect any drift, identifying whether the model's accuracy has decreased significantly compared to past performance. If drift is detected the workflow moves forward by running a SageMaker Training job to train or update a fraud-detection model with the latest data.

After training, another Processing job evaluates the newly trained "challenger" model against the current "champion" model using a common test dataset. Based on this evaluation, the pipeline automatically selects and deploys the superior model to a SageMaker endpoint, ensuring the live system consistently maintains the highest possible accuracy.

This fully automated pipeline ensures that models stay robust, up-to-date, and continuously improve over time with minimal manual intervention.

2. Requirements

- Basic understanding of AWS Lambda, S3, Step Functions, and SageMaker is required to follow the workflow and setup resources correctly.
- Ability to write and modify Python scripts, especially for handling AWS interactions using the boto3 library.
- Familiarity with writing task configurations similar to Docker-style job definitions for SageMaker Processing and Training jobs

3. Procedure

Step 1: Create S3 Buckets and Folder Structure

The first step is to set up the necessary S3 storage for your pipeline. You will need three separate S3 buckets to organize datasets, model artifacts, and scripts.

1. Create three new S3 buckets with unique names:

- your-datasets-bucket – for all datasets
- your-models-bucket – for staging trained models
- your-scripts-bucket – for your Python scripts

2. In your-datasets-bucket, create three “folders” (prefixes):

- raw/ – store raw (unprocessed) data
- processed/ – store cleaned or transformed data
- test/ – store test datasets

Step 2: Create & Configure a Lambda Function for Raw-Data Uploads

The next step is to create a Lambda function that automatically triggers your Step Functions workflow whenever new data is uploaded to S3.

1. Create a Lambda function and ensure that the function has permissions to access S3.

2. In your datasets S3 bucket, configure an event notification to trigger the Lambda whenever a new object is uploaded to the raw/ folder. Set the event type to PUT and specify the prefix accordingly. Choose your Lambda function as the destination.

Step 3: Define & Integrate the State Machine

1. Create a new Step Functions state machine to orchestrate the pipeline.

- Open the AWS Management Console and navigate to Step Functions → State machines → Create state machine.
- Choose **Standard** as the workflow type and author the definition from scratch.
- As a starting point, you can create a simple Pass state to test that executions are triggered correctly.

2. Update Your Lambda Function to Start the State Machine

Update the Lambda handler provided below by filling in the missing parts marked as TODO. You need to extract the uploaded file name from the S3 event, define your datasets bucket name, and construct the S3 URIs for raw input, processed output, and test data. Then, generate a unique job ID by sanitizing the file name and appending a UTC timestamp. Update the s3StagingUri and state_machine_arn fields with the correct values for your setup. Finally, ensure that your Lambda function has permission to start executions on your Step Functions state machine.

1

2

3

4

5

6

7

8

9

10

```
37
38
39
40
import json
import boto3
import re
from datetime import datetime


def lambda_handler(event, context):
    stepfunctions_client = boto3.client('stepfunctions')


    file_name = # TODO: Extract the file name from the event


    # Define your datasets bucket name (adjust accordingly)
    datasets_bucket = TODO: Replace with your actual datasets bucket


    # Build the required S3 URIs
    s3_input_uri = f"s3://{datasets_bucket}/raw/{file_name}"        # Raw input file URI
    s3_processed_uri = f"s3://{datasets_bucket}/processed/"         # Processed data output fo
lder
    s3_test_uri = f"s3://{datasets_bucket}/test/"                # Test data folder


    # Generate a unique job ID
    # TODO: combine a sanitized version of file_name with the current UTC timestamp
    unique_job_id = None  # ← implement this
```

```
    # Create the payload to pass to Step Functions
    payload = {
        "id": unique_job_id,
        "s3InputUri": s3_input_uri,
        "s3ProcessedOutputUri": s3_processed_uri,
        "s3TestDataUri": s3_test_uri,
        "s3StagingUri": "s3://your-models-
bucket/staging/",   # TODO: Update with your models bucket staging URI
        "fileName": file_name
    }

    # Start the Step Functions execution
    state_machine_arn = # TODO: Update with your Step Function ARN

    response = stepfunctions_client.start_execution(
        stateMachineArn=state_machine_arn,
        input=json.dumps(payload)
    )
```

3. End-to-End Testing

Perform an end-to-end test to verify that everything is connected correctly. First, upload a test file into the raw/ folder of your datasets S3 bucket. This should automatically trigger your Lambda function.

Next, check the Lambda's CloudWatch logs to confirm that the function executed successfully. You should see logs printing the constructed S3 URIs and a Step Functions StartExecution response. Finally, open the Step Functions console and verify that a new execution has started and completed successfully.

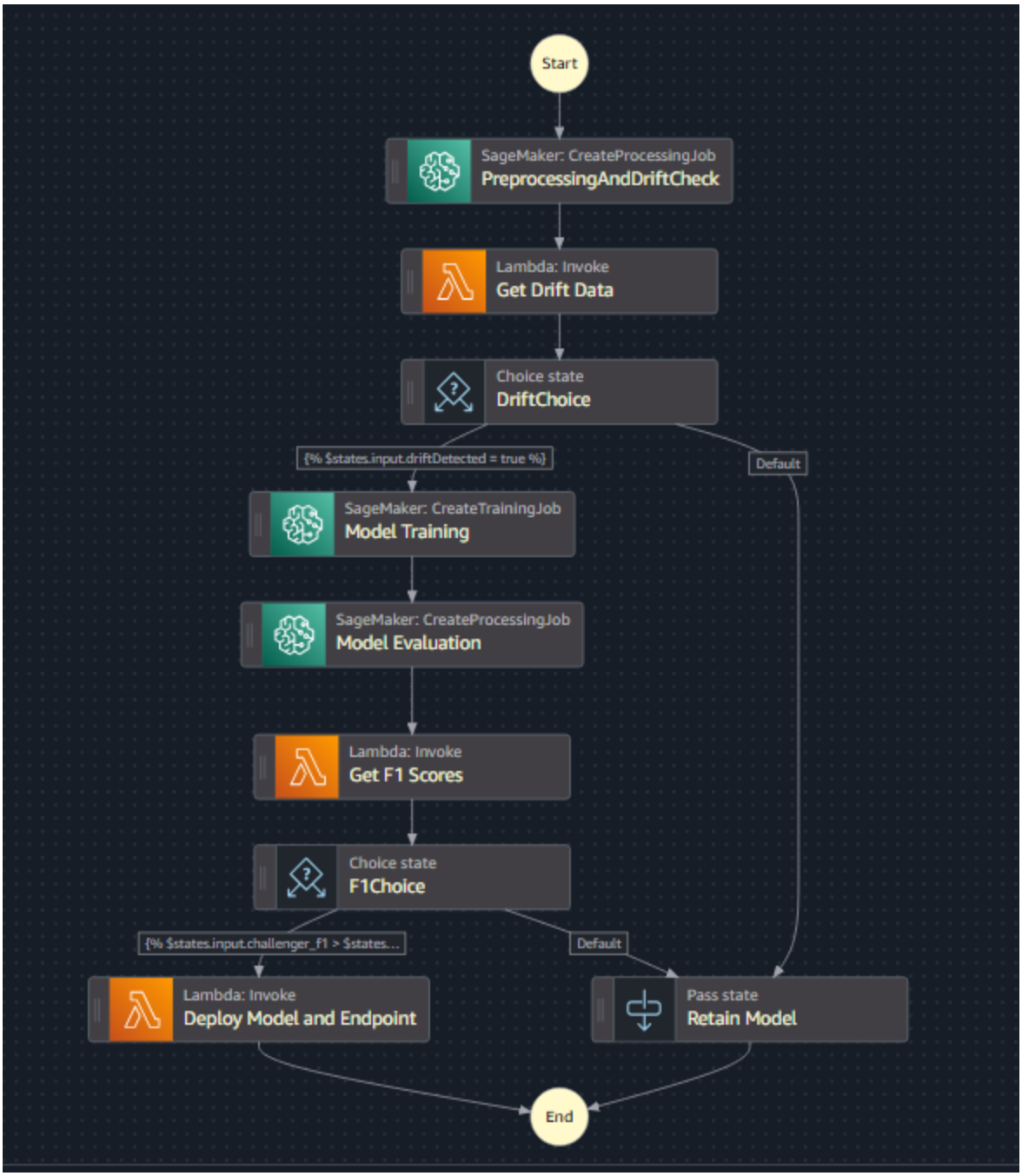
Step 4: State Machine Definition

4.1 Step Function Initialization

Open your MP13 state machine in Step Functions and switch to the Edit view. Update the workflow to match the flow shown in the figure below.

Tip: You can build the flow incrementally by adding one task at a time and use the "Test State" option to validate each steps before connecting the states.

The pipeline should start with a preprocessing and drift detection step, followed by drift decision logic. If drift is detected, the model is retrained, evaluated, and compared against the current model before deciding whether to deploy the new model. Otherwise, the existing model is retained.



Create a new IAM role for SageMaker that has permission to access your S3 buckets and launch SageMaker Processing and Training jobs.

4.2 Add “Preprocessing And Drift Check” Task (CreateProcessingJob)

In this step, you will set up a SageMaker Processing Job that runs the drift_preproc.py script to perform preprocessing and drift detection on incoming data.

First, manually create a SageMaker Processing Job through the Amazon SageMaker console.

- Use the container image:
    - 683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:0.20.0-cpu-py3 *(We use this image because it supports lightweight Python scripts like drift_preproc.py and is optimized for small, fast preprocessing tasks.)*
- Attach your SageMaker IAM role that has permission to access your S3 buckets.
- Set the input source as your scripts bucket (your-scripts-bucket) and mount it inside the container so that the script drift_preproc.py is accessible.

In the container arguments, make sure to pass all necessary parameters to the script:

- --raw-data-uri
- --evaluation-data-uri
- --processed-data-uri
- --file-name
- --staging

For the output, specify your models bucket staging folder, where the processing outputs like transformed datasets and drift results will be saved.

Note: In SageMaker, anything you specify in inputs gets copied into the corresponding local container path inside the processing job. Similarly, after the script finishes running, any files saved to the specified local output folder inside the container will automatically get uploaded back to the linked S3 output URI.

The drift_preproc.py script will:

- Save the processed training data to your processed folder.
- Update and save the test dataset to your test folder.
- Upload the drift detection results (drift_result.json) to your staging area.

Once you manually verify that the processing job works correctly, you can automate it by defining a CreateProcessingJob state inside your Step Function like this:

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

```
28
29
30
31
32
33
34
35
36
37
38
39
40
{
  "ProcessingJobName": "# TODO: Provide a unique job name",
  "RoleArn": "# TODO: Provide your SageMaker IAM role ARN",
  "AppSpecification": {
    "ImageUri": "683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:0.23-1-cpu-py3",
    "ContainerEntrypoint": ["python3"],
    "ContainerArguments": [
      "# TODO: Provide the path to your drift_preproc.py script",
      "--raw-data-uri", "# TODO: Pass the input raw S3 URI",
      "--evaluation-data-uri", "# TODO: Pass the evaluation S3 URI",
      "--processed-data-uri", "# TODO: Pass the processed output S3 URI",
      "--file-name", "# TODO: Pass the input file name",
```

```
    "--staging", "# TODO: Pass the S3 URI of your models/staging folder"
   ]
  },
  "ProcessingResources": {
   "ClusterConfig": {
    "InstanceCount": 1,
    "InstanceType": "ml.t3.medium",
    "VolumeSizeInGB": 10
   }
  },
  "ProcessingInputs": [
   {
    "InputName": "# TODO: Name your input (e.g., 'scripts')",
    "S3Input": {
     "S3Uri": "# TODO: Provide the S3 URI for your scripts folder",
     "LocalPath": "# TODO: Provide the local container path where your scripts will be copie
d",
     "S3DataType": "S3Prefix",
     "S3InputMode": "File"
    }
   }
  ],
  "ProcessingOutputConfig": {
   "Outputs": [
    {
     "OutputName": "output",
```

```
    "S3Output": {
      "S3Uri": "# TODO: Provide the S3 URI where outputs should be saved",
      "LocalPath": "/opt/ml/processing/output",
```

4.3. Create Lambda 2: Fetch Evaluation JSON from S3

This Lambda function is responsible for fetching the evaluation results generated in the previous step of the pipeline. After the model evaluation script runs, it saves a JSON file into S3 that contains important metrics such as whether drift was detected (driftDetected) and the F1 scores on the old dataset and the combined dataset (old dataset + new dataset).

The Lambda retrieves this evaluation JSON file from S3, parses its content, and passes the extracted data back into the Step Functions workflow. This allows the state machine to make decisions based on the evaluation results, such as whether to retrain the model or keep the current one.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

```
import json
import boto3
import logging


# Set up logging
```

```
logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Initialize S3 client
s3_client = boto3.client('s3')

def lambda_handler(event, context):
  try:
    # TODO: Get the S3 link from the event payload
    s3_link =

    # TODO: Extract bucket and key from the S3 link
    bucket, key =

    # TODO: Fetch the object from S3
    response =

    # TODO: Read the object content and decode it
    content =

    #Parsing the content as JSON
    data = json.loads(content)

    # Log and return the parsed JSON
    logger.info("Returning JSON content: %s", data)
    return data
```

```
except Exception as e:
    logger.error("Error processing the S3 JSON: %s", str(e))
    raise e
```

After the Lambda invocation, create a **Choice state** that checks the driftDetected variable returned from the evaluation JSON.

- If driftDetected is true, it indicates that the production model did not exist or did not perform well on the new dataset, and a retraining process should be triggered to build a new model.
- If driftDetected is false, it means there was no significant performance drop, and the workflow should continue along the default path without making any changes, retaining the existing production model.

4.4 Add "Model Training" Task (CreateTrainingJob)

In this step, you will configure a **SageMaker Training Job** that runs the train.py script to train or retrain your fraud detection model.

First, manually test creating a **Training Job** through the SageMaker console:

- Use the same container image as Preprocessing step.
- As **input**, mount two folders:

In the **container entrypoint and arguments**, specify:

- The Python entrypoint as python3
- The script path /opt/ml/input/data/script/train.py
- And the required script arguments: processed dataset file name and staging folder URI.

The **train.py** script will:

- Load the processed training data,
- Attempt to warm-start training from the existing production model if available (otherwise train from scratch),

- Save the trained model, imputer, scaler, and inference script,
- Package everything into a model.tar.gz artifact for deployment.

For **outputs**, configure SageMaker to save the results (the model artifact) into your **staging S3 folder**.

Note: In SageMaker Training Jobs:

- **InputDataConfig** maps your S3 datasets and scripts into the container's /opt/ml/input/data/<channel_name> folders.
- **OutputDataConfig** ensures any files placed under /opt/ml/model in the container will be automatically uploaded to the configured S3 URI after training finishes.

After verifying that your training job runs successfully, automate it inside your Step Functions workflow by defining a **CreateTrainingJob** task using the following template:

1

4.5 Add "Model Evaluation" Task (CreateProcessingJob)

4.6 Attach Lambda 2: Fetch Champion F1 and Challenger F1 scores

Use the Lambda function 2 defined in step 4 point 4 to fetch the evaluation JSON file produced by the model evaluation step. This file contains important metrics, specifically the champion_f1 and challenger_f1 scores. Configure the Lambda to read and parse this JSON file from S3 and return the extracted information to the Step Functions workflow.

After this Lambda invocation, create a Choice state that compares the F1 scores.

- If the challenger_f1 is greater than the champion_f1, the workflow should proceed to deploy the new model and update the SageMaker endpoint.
- Otherwise, the workflow should follow the default path and retain the existing model without making any changes.

This logic ensures that only genuinely better models are promoted to production automatically.

4.7 Create Lambda 3: Package & Deploy to SageMaker

Create Lambda function 3 which is responsible for deploying the newly trained model to SageMaker for real-time inference. It first deletes any existing model and endpoint configuration if they exist, then creates a new model, sets up a fresh endpoint

configuration, and either updates an existing endpoint or creates a new one. This ensures that the best available model is always live and ready to serve predictions.

- Extract the model_url from the incoming event payload: This provides the location of the newly trained model artifact (model.tar.gz) in S3, which SageMaker needs to deploy the model.
- Replace the role_arn placeholder with your SageMaker execution role ARN.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

```
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
import json
import boto3

def lambda_handler(event, context):
    sagemaker_client = boto3.client('sagemaker')

    model_url = # Extract only the model URL from the event
```

```
model_name = "mp13-production-model"
endpoint_name = "mp13-production-endpoint"
endpoint_config_name = "mp13-production-endpoint-config"
role_arn = # TODO: Replace with your SageMaker role ARN

# Step 1: Delete existing model if it exists
try:
    sagemaker_client.delete_model(ModelName=model_name)
    print(f"Deleted existing model: {model_name}")
except Exception as e:
    print(f"Model doesn't exist or couldn't be deleted: {str(e)}")

# Step 2: Create a new model
try:
    sagemaker_client.create_model(
        ModelName=model_name,
        ExecutionRoleArn=role_arn,
        PrimaryContainer={
            "Image": "683313688378.dkr.ecr.us-east-1.amazonaws.com/sagemaker-scikit-learn:0.23-1-cpu-py3",
            "ModelDataUrl": model_data_url,
            "Environment": {
                "SAGEMAKER_PROGRAM": "inference.py",
                "SAGEMAKER_SUBMIT_DIRECTORY": "/opt/ml/model"
            }
```

```
        }
    )
    print(f"Created new model: {model_name}")
except Exception as e:
    print(f"Error creating model: {str(e)}")
    raise e


# Step 3: Delete existing endpoint config if it exists
```

Step 5: Testing on Provided Datasets

You are given three datasets: base1_dataset.csv, base2_dataset.csv, concept_drift_dataset.csv

The datasets base1_dataset.csv and base2_dataset.csv are from the same distribution and should not cause drift, while concept_drift_dataset.csv has significant changes compared to the earlier ones and should trigger drift detection.

Upload each of these datasets one by one to the raw/ folder in your datasets S3 bucket, and observe how your Step Functions state machine executes.

Part 1: Testing with base1_dataset.csv

- Upload base1_dataset.csv to the raw/ folder in your datasets bucket.
- The Step Function should execute the following flow: Start Preprocessing And Drift Check → Get Drift Data → DriftChoice → Model Training → Model Evaluation → Get F1 Scores → F1 Choice → Deploy Model and Endpoint → End

This happens because some drift is detected since there is no model on production and which triggers training and creates a model and an endpoint.

Part 2: Testing with base2_dataset.csv

- Upload base2_dataset.csv to the raw/ folder.
- The Step Function should execute the following flow: Preprocessing And Drift Check → Get Drift Data → DriftChoice → Retain Model → End

Here, since the dataset is similar to the previous base dataset, **no drift is detected**, and the system decides to retain the current production model without retraining or redeploying.

Part 3: Testing with concept_drift_dataset.csv

- Upload concept_drift_dataset.csv to the raw/ folder.
- The Step Function should execute: Start Preprocessing And Drift Check → Get Drift Data → DriftChoice → Model Training → Model Evaluation → Get F1 Scores → F1 Choice → Deploy Model and Endpoint → End

In this case, **driftDetected will be true** because the production model fails to maintain good performance when evaluated on the combined old and new datasets compared to its original performance on just the old data. As a result, the pipeline retrains a new model on the updated data and, if the new model performs better, it updates the SageMaker endpoint to serve the improved model.

Step 7: Testing on Provided Datasets

In the Step Functions console, locate the Execution ARN for each successful run for Part 1, Part 2, and Part 3. Copy the ARNs for all three runs and paste them into the provided submit.py file to complete your submission.

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue